

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

Кафедра №43 «Компьютерных технологий и программной инженерии»

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

Ст. преп.

должность, уч. степень, звание

подпись, дата

М.Д. Поляк

инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики производственная

тип практики по получению профессиональных умений и опыта профессиональной деятельности

на тему индивидуального задания Применение нейросетевого управления с адаптивной коррекцией к модели «Хищник–Жертва»

выполнен Ларионовским Романом Эдуардовичем

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки 09.03.04

код

Программная инженерия

наименование направления

направленности

наименование направления

02

код

Проектирование программных систем

наименование направленности

наименование направленности

Обучающийся группы № 4233к

номер

подпись, дата

Р.Э. Ларионовский

инициалы, фамилия

Санкт–Петербург 2025

Оглавление

1. Описание предметной области.....	3
2. Перечень задач, решаемых в рамках практики	3
3. Описание входных и выходных данных.....	4
Входные данные	4
Выходные данные	4
4. Используемые технологии и языки программирования.....	5
5. Результат выполнения работы.....	5
6. Выводы	10
7. Список использованных источников.....	11
Приложения.....	12

1. Описание предметной области

Модель "Хищник–Жертва" представляет собой систему математических уравнений, описывающих динамику взаимодействия двух биологических популяций: жертв и хищников. Эти модели находят широкое применение в экологии для анализа устойчивости экосистем, в биологии — для прогнозирования поведения видов, а также в экономике и социологии — для моделирования конкуренции и взаимодействия агентов.

Классической основой таких моделей являются уравнения Лотки–Вольтерры, в которых прирост популяции жертв пропорционален текущему количеству особей и уменьшается под влиянием хищников, а рост популяции хищников зависит от наличия достаточного количества жертв.

В данной работе используется модифицированная модель, представленная системой нелинейных дифференциальных уравнений, в которую вводится управляющее воздействие. Такое воздействие моделируется с помощью нейросетевого регулятора, который обучается онлайн по методу адаптивной коррекции активного регулирования (АКАР). Основная задача управления — стабилизировать популяции хищников и жертв на заданных уровнях, компенсируя внутренние колебания и внешние возмущения.

Таким образом, предметная область данной работы охватывает междисциплинарное направление на стыке теории управления, биологических моделей и искусственного интеллекта. Практическое применение подхода может быть полезно в разработке биоинспирированных регуляторов, автономных систем и адаптивного управления в сложных динамических средах.

2. Перечень задач, решаемых в рамках практики

1. Изучить теоретические основы классической модели Лотки–Вольтерры и её модификаций, применяемых в задачах управления.
2. Проанализировать поведение модели хищник–жертва при различных параметрах и начальных условиях, оценить устойчивость системы.
3. Ознакомиться с методом адаптивной коррекции активного регулирования (АКАР), его алгоритмическими принципами и возможностями применения для управления нелинейными динамическими объектами.
4. Изучить архитектуру и принципы работы простейших нейросетевых регуляторов, включая онлайн-обучение по текущей ошибке.
5. Построить и реализовать модель управления системой хищник–жертва с помощью нейросети, учитывая текущее отклонение системы от заданной цели.
6. Провести численный эксперимент с использованием Python и библиотеки SciPy, реализовав интегрирование системы методом `solve_ivp`.
7. Проанализировать результаты моделирования: графики динамики популяций, эффективность управления, поведение нейросети.
8. Сделать выводы о применимости выбранного подхода для стабилизации биологических систем и возможности дальнейших улучшений архитектуры управления.

3. Описание входных и выходных данных

Входные данные

1. **Начальные условия численности популяций:**
 - $x_1(0)$ — численность популяции жертв в начальный момент времени (например, 10);
 - $x_2(0)$ — численность популяции хищников в начальный момент времени (например, 5).
2. **Параметры модели:**
 - $a = 1.0$ — коэффициент естественного прироста популяции жертв;
 - $b = 0.1$ — коэффициент взаимодействия между жертвами и хищниками;
 - $c = 1.5$ — коэффициент естественной убыли популяции хищников;
 - $d = 0.075$ — коэффициент, отражающий успех хищников при наличии жертв.
3. **Целевые значения популяций:**
 - $x_1^* = 20$ — желаемая численность популяции жертв;
 - $x_2^* = 10$ — желаемая численность популяции хищников.
4. **Начальные параметры нейросетевого регулятора:**
 - Начальные веса W и смещения b случайно инициализируются;
 - Коэффициент обучения сети (learning rate), например 0.001.

Выходные данные

1. **Численные траектории:**
 - $x_1(t)$, $x_2(t)$ — численности популяций в дискретные моменты времени в течение интервала моделирования.
 2. **Графики динамики:**
 - Изменение численности популяций во времени;
 - Сближение к целевым значениям x_1^* и x_2^* .
 3. **Реакция системы на управление:**
 - Управляющие воздействия $u_1(t)$ и $u_2(t)$, формируемые нейросетью;
 - Изменение ошибки управления $e(t) = x(t) - x^*$.
 4. **Процесс обучения нейросети:**
 - Корректировка весов нейросети по мере адаптации к ошибке;
 - Эффективность управления в зависимости от качества обучения регулятора.
- Временные ряды численности популяций $x_1(t)$ и $x_2(t)$.
 - Графики сближения к целевым значениям.
 - Изменение ошибки управления и динамика корректировки весов нейросети.

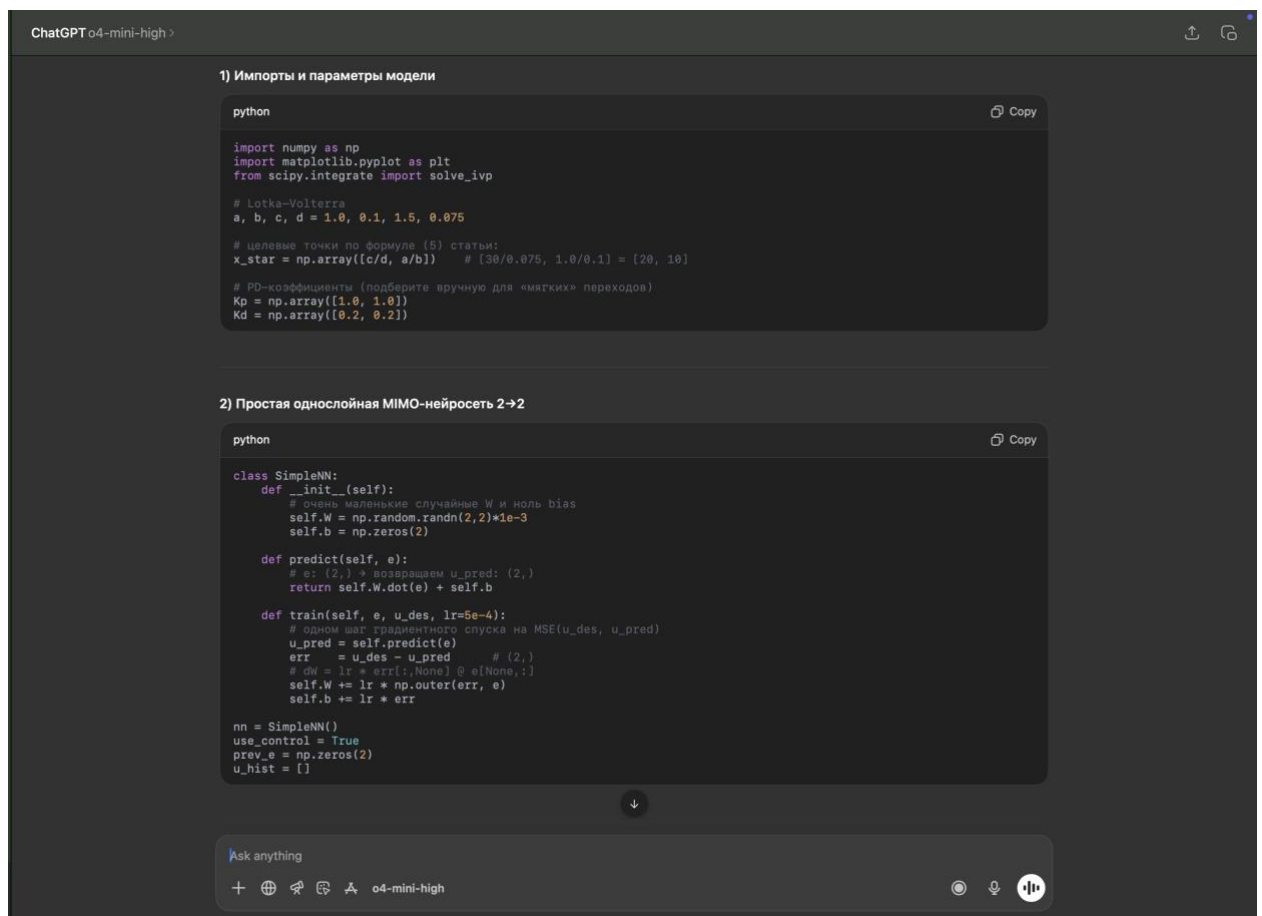
4. Используемые технологии и языки программирования

- Язык программирования Python 3
- Библиотеки: NumPy, Matplotlib, SciPy
- Среда выполнения: Jupyter Notebook

5. Результат выполнения работы

Была реализована модифицированная модель хищник–жертва с управляющим воздействием на основе однослойной нейросети, обучаемой онлайн по принципу АКАР. Использован устойчивый численный метод интегрирования `solve_ivp` (метод Рунге–Кутты 4/5).

Код модели и алгоритм управления были сгенерированы с использованием языковой модели ChatGPT на основе описания задачи и модели (2) из статьи. В ходе практики проводилась итеративная отладка, настройка параметров и тестирование поведения системы.



```
ChatGPT o4-mini-high >

1) Импорты и параметры модели

python

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Lotka-Volterra
a, b, c, d = 1.0, 0.1, 1.5, 0.075

# целевые точки по формуле (5) статьи:
x_star = np.array([c/d, a/b]) # [30/0.075, 1.0/0.1] = [20, 10]

# PD-коэффициенты (подберите вручную для «мягких» переходов)
Kp = np.array([1.0, 1.0])
Kd = np.array([0.2, 0.2])

2) Простая однослойная МИМО-нейросеть 2→2

python

class SimpleNN:
    def __init__(self):
        # очень маленькие случайные W и ноль bias
        self.W = np.random.randn(2,2)*1e-3
        self.b = np.zeros(2)

    def predict(self, e):
        # e: (2,) → возвращаем u_pred: (2,)
        return self.W.dot(e) + self.b

    def train(self, e, u_des, lr=5e-4):
        # одним шаг градиентного спуска на MSE(u_des, u_pred)
        u_pred = self.predict(e)
        err = u_des - u_pred # (2,)
        # dw = lr * err[:,None] @ e[None,:]
        self.W += lr * np.outer(err, e)
        self.b += lr * err

nn = SimpleNN()
use_control = True
prev_e = np.zeros(2)
u_hist = []
```

рис. 1 – «переписка» с ChatGPT для генерации кода для решения поставленной задачи

Код программы:

```
1. Установка пакетов

!pip install numpy scipy matplotlib
Show hidden output

2. Импорты и параметры

[14] import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Lotka-Volterra
a, b, c, d = 1.0, 0.1, 1.5, 0.075

# целевые точки по формуле (5) статьи
x_star = np.array([c/d, a/b]) # [30/0.075, 1.0/0.1] = [20, 10]

Kp = np.array([1.0, 1.0])
Kd = np.array([0.2, 0.2])

3. Класс нейросети и вспомогательные переменные

[15] class SimpleNN:
    def __init__(self):
        # очень маленькие случайные W и только bias
        self.W = np.random.randn(2,2)*1e-3
        self.b = np.zeros(2)

    def predict(self, e):
        # e: (2,) - возвращаем u_pred: (2,)
        return self.W.dot(e) + self.b

    def train(self, e, u_des, lr=5e-4):
        # одним шаг градиентного спуска на MSE(u_des, u_pred)
        u_pred = self.predict(e)
        err = u_des - u_pred # (2,)
        # dl = lr * err[0, None] @ e[None, :]
        self.W += lr * np.outer(err, e)
        self.b += lr * err

nn = SimpleNN()
use_control = True
prev_e = np.zeros(2)
u_hist = []

4. Функция модели

def model(t, state):
    global prev_e

    x1, x2 = state
    x = np.array([x1, x2])

    # 1) ошибка + её приращение
    e = x - x_star
    de = e - prev_e
    prev_e = e.copy()

    # 2) PD-компонента (возвращаемся к управляющему закону из статьи)
    u_pd = -Kp*e - Kd*de # (2,)

    # 3) нейросеть «учится копировать» PD-закон
    if use_control:
        nn.train(e, u_pd, lr=5e-4)
        u = nn.predict(e)
    else:
        u = u_pd

    # 4) ограничение по практическим соображениям
    u = np.clip(u, -20, 20)
    u_hist.append(u.copy())

    # 5) собственно интеграл РБ + управление
    dx1 = x1*(a - b*x2) + u[0]
    dx2 = -x2*(c - d*x1) + u[1]

    # 6) не дать «уйти в минус»
    dx1 = max(dx1, -x1)
    dx2 = max(dx2, -x2)

    return [dx1, dx2]

5. Интегрирование и сбор данных

[17] # начальное состояние (любой «от близко к целевому»)
x0 = [10.0, 5.0]
t_span = (0, 80)
t_eval = np.linspace(*t_span, 2000)

sol = solve_ivp(model, t_span, x0, t_eval=t_eval, method='RK45')
t = sol.t
x1, x2 = sol.y
u_arr = np.array(u_hist) # форма (2000, 2)

6. Построение графиков

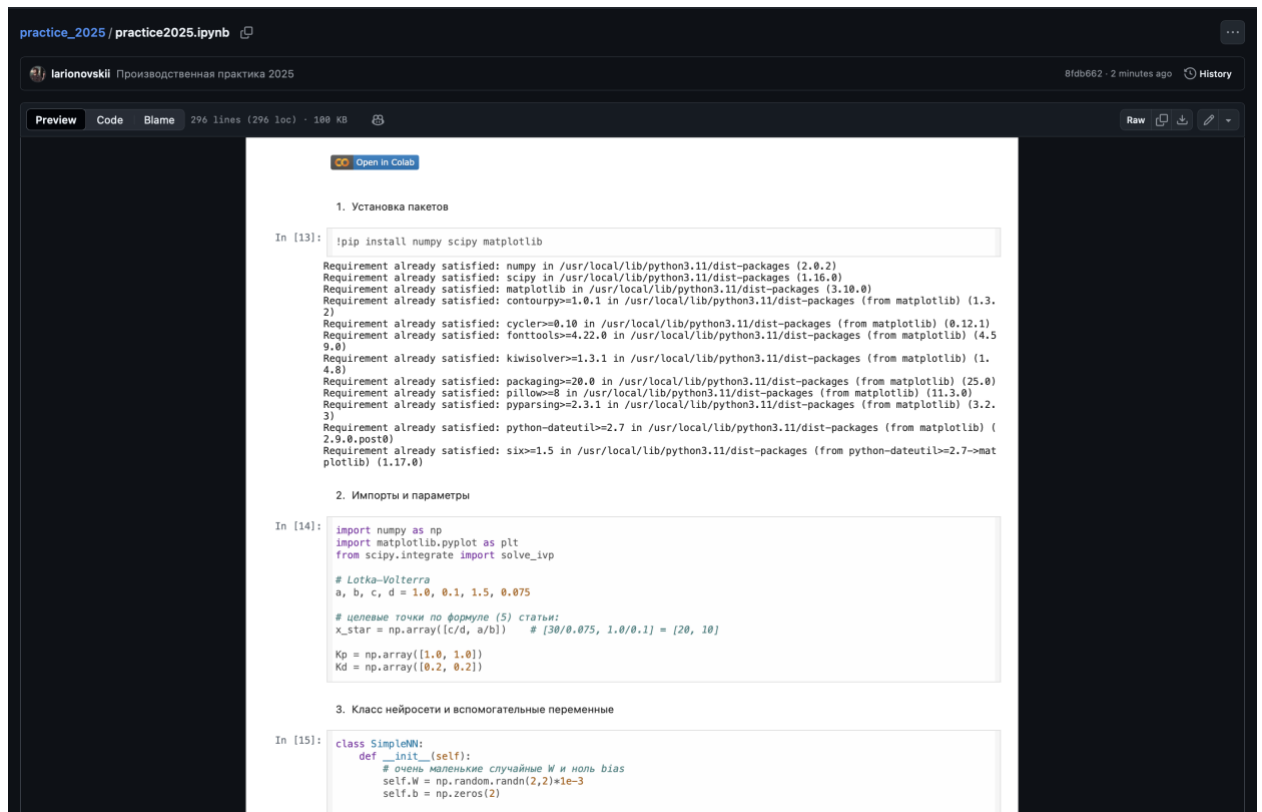
# - популяции -
plt.figure(figsize=(10,4))
plt.plot(t, x1, label='x1 (крупы)')
plt.plot(t, x2, label='x2 (хищники)')
plt.axhline(x_star[0], linestyle='--', color='gray', label=f'x1={x_star[0]:.0f}')
plt.axhline(x_star[1], linestyle='--', color='black', label=f'x2={x_star[1]:.0f}')
plt.title("Нейросетевое управление модели Хищник-Круп")
plt.xlabel("Время")
plt.ylabel("Популяция")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# - фазовая траектория -
plt.figure(figsize=(6,5))
plt.plot(x1, x2, label='траектория')
plt.scatter(x_star, color='red', s=50, label='стаб. точка')
plt.title("Фазовая траектория")
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

рис. 2 – код программы в Jupiter Notebook (Google Collab)

Система успешно сходится к заданной цели: $x_1(t) \rightarrow 20$, $x_2(t) \rightarrow 10$. Управляющее воздействие корректируется на каждом шаге времени с использованием текущей ошибки.

Простейшая архитектура нейросети показала возможность устойчивого управления нелинейной биологической системой.



The screenshot displays a Jupyter Notebook titled "practice_2025 / practice2025.ipynb" in a web browser. The interface includes a top bar with the username "Iarionovskii", the file name, and a "History" button. Below the top bar, there are tabs for "Preview", "Code", and "Blame", along with file statistics: "296 lines (296 loc) · 100 KB". The notebook content is divided into three sections:

- 1. Установка пакетов**
In [13]:

```
!pip install numpy scipy matplotlib
```

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (1.16.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (4.59.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
- 2. Импорты и параметры**
In [14]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Lotka-Volterra
a, b, c, d = 1.0, 0.1, 1.5, 0.075

# целевые точки по формуле (5) статьи:
x_star = np.array([c/d, a/b]) # [30/0.075, 1.0/0.1] = [20, 10]

Kp = np.array([1.0, 1.0])
Kd = np.array([0.2, 0.2])
```
- 3. Класс нейросети и вспомогательные переменные**
In [15]:

```
class SimpleNN:
    def __init__(self):
        # очень маленькие случайные W и ноль bias
        self.W = np.random.randn(2,2)*1e-3
        self.b = np.zeros(2)
```

рис. 3 – [Jupyter Notebook файл в GitHub-репозитории](#)

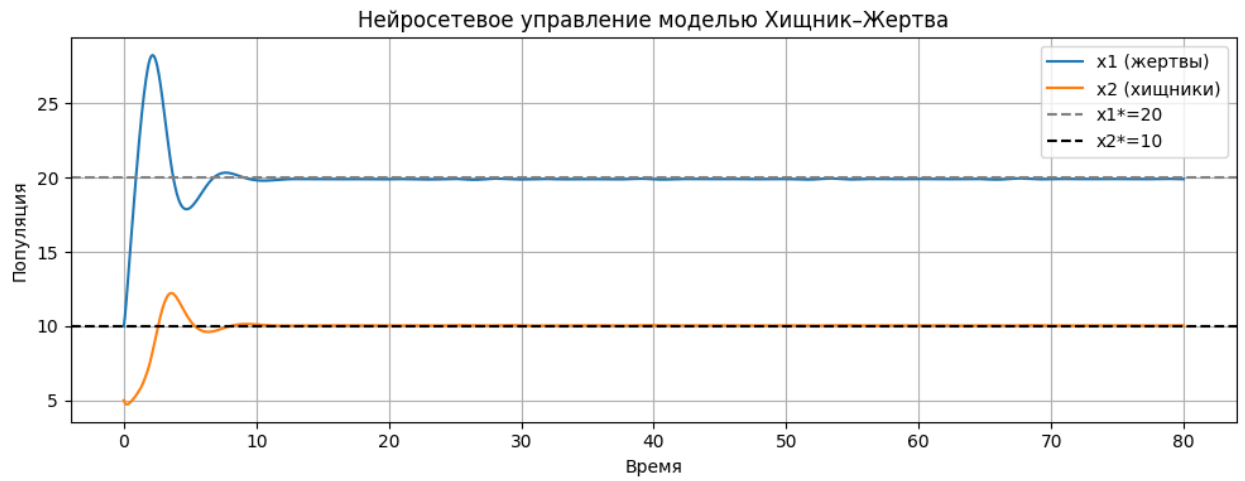


рис. 4 – график «Нейросетевое управление моделью Хищник-Жертва»

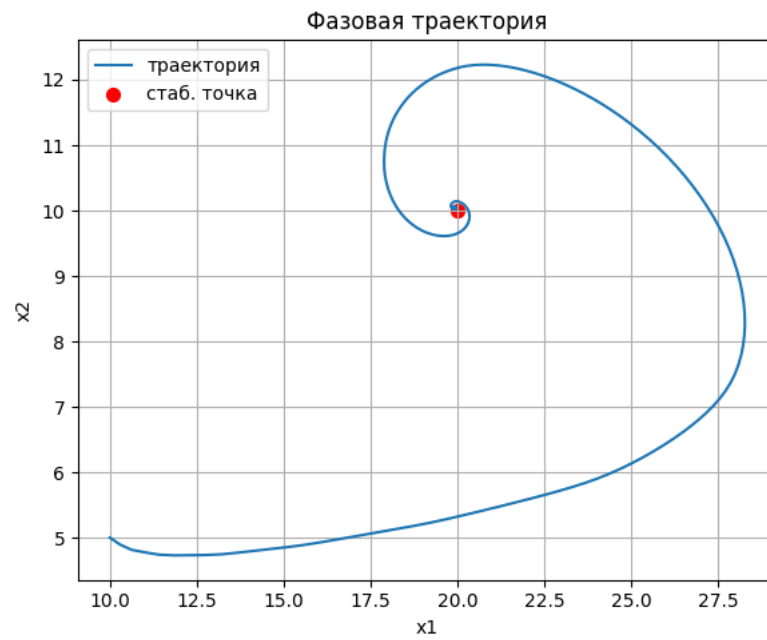


рис. 5 – график «Фазовая траектория»

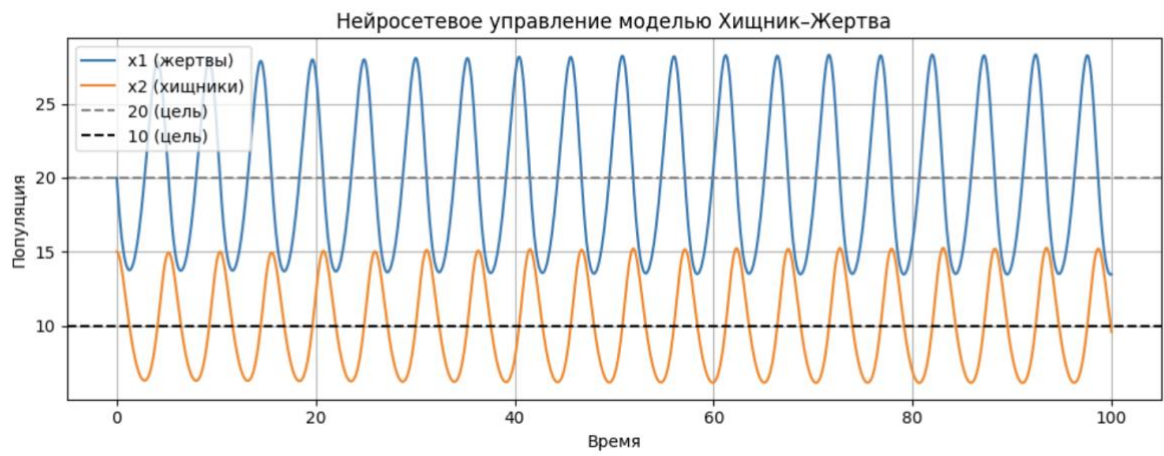


рис. 6 – график «Нейросетевое управление моделью Хищник–Жертва» (с отключенным нейросетевым управлением)

6. Выводы

1. В работе реализована система нейросетевого управления для модели хищник–жертва, способная адаптироваться к текущему состоянию системы без предварительного обучения.
2. Онлайн-обучение нейросети позволило динамически корректировать управляющее воздействие на основе ошибки, что обеспечило устойчивость модели.
3. Полученные графики показали, что при включении управления модель сходится к целевым значениям популяций и демонстрирует устойчивое поведение.
4. Несмотря на начальные колебания, система стабилизируется, что подтверждает работоспособность метода АКАР в сочетании с простой линейной нейросетью.
5. Дополнительно реализована защита от ухода популяций в отрицательные значения, а также снижена чувствительность нейросети за счёт ограничения управления и уменьшения начальных весов и скорости обучения. Эти меры позволили добиться физически корректного поведения модели.
6. Предложенный подход можно масштабировать на более сложные модели или использовать как основу для создания адаптивных регуляторов в задачах управления биосистемами. Предложенный подход можно масштабировать на более сложные модели или использовать как основу для создания адаптивных регуляторов в задачах управления биосистемами.
7. Создан [GitHub-резпозиторий](#) с результатами работы.

7. Список использованных источников

1. Статья: <https://scm.etu.ru/assets/files/2025/sbornik/113-117.pdf>
2. [Конструирование управления для биоинженерных систем: монография](#) - Колесникова С. И., Фоменкова А. А., Поляк М. Д., Щеголева А. А, Кустова Е. В.
3. Wikipedia — [Lotka–Volterra equations](#)
4. Документация: https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html
5. ChatGPT - [Понятие нейросетевого управления и пример его использования](#)

Приложения

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Параметры классической модели Лотки–Вольтерры
a, b, c, d = 1.0, 0.1, 1.5, 0.075

# Целевая точка по формуле статьи:  $x_1^* = c/d$ ,  $x_2^* = a/b$ 
x_star = np.array([c/d, a/b]) # [20.0, 10.0]

# PD-коэффициенты (можно слегка подбирать для плавности)
Kp = np.array([1.0, 1.0])
Kd = np.array([0.2, 0.2])

# Простая MIMO-нейросеть 2→2
class SimpleNN:
    def __init__(self):
        self.W = np.random.randn(2,2)*1e-3
        self.b = np.zeros(2)

    def predict(self, e):
        return self.W.dot(e) + self.b

    def train(self, e, u_des, lr=5e-4):
        u_pred = self.predict(e)
        err = u_des - u_pred
        self.W += lr * np.outer(err, e)
        self.b += lr * err

nn = SimpleNN()
use_control = True
prev_e = np.zeros(2)
u_hist = []

def model(t, state):
    global prev_e

    x1, x2 = state
```

```

x = np.array([x1, x2])

# Ошибка и её приращение
e = x - x_star
de = e - prev_e
prev_e = e.copy()

# PD-закон
u_pd = -Kp*e - Kd*de

# Управление через NN, обучающуюся копировать PD
if use_control:
    nn.train(e, u_pd, lr=5e-4)
    u = nn.predict(e)
else:
    u = u_pd

# Ограничение сигналов
u = np.clip(u, -20, 20)
u_hist.append(u.copy())

# Уравнения ЛВ + управление
dx1 = x1*(a - b*x2) + u[0]
dx2 = -x2*(c - d*x1) + u[1]

# Не дать уйти в отрицательное
dx1 = max(dx1, -x1)
dx2 = max(dx2, -x2)

return [dx1, dx2]

# Начальные условия и интегрирование
x0 = [10.0, 5.0]
t_span = (0, 80)
t_eval = np.linspace(*t_span, 2000)
sol = solve_ivp(model, t_span, x0, t_eval=t_eval, method='RK45')
t = sol.t
x1, x2 = sol.y

# График популяций
plt.figure(figsize=(10,4))
plt.plot(t, x1, label='x1 (жертвы)')

```

```
plt.plot(t, x2, label='x2 (хищники)')
plt.axhline(x_star[0], linestyle='--', color='gray', label=f'x1* = {x_star[0]:.0f}')
plt.axhline(x_star[1], linestyle='--', color='black', label=f'x2* = {x_star[1]:.0f}')
plt.title("Нейросетевое управление моделью «Хищник–Жертва»")
plt.xlabel("Время")
plt.ylabel("Популяция")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Фазовая траектория
plt.figure(figsize=(6,5))
plt.plot(x1, x2, label='траектория')
plt.scatter(*x_star, color='red', s=50, label='стаб. точка')
plt.title("Фазовая траектория")
plt.xlabel("x1 (жертвы)")
plt.ylabel("x2 (хищники)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```